Two Pointers, Sliding Window

1) Reverse a String

Input s = ["h", "e", "l", "l", "o"]

```
void reverseString(vector<char>& s){
    int start = 0;
    int end = s.size()-1;
    int temp;

    while(start<end){
        temp = s[start];
        s[start] = s[end];
        s[end] = temp;

        start++;
        end--;
    }
}
```

Output

olleh

PAGE No.:
DATE:

2) Valid Palindrome

Input s = "A man, a plan, a canal: Panama"
Output → True.

```
bool isPalindrome (string s) {
    int start = 0, end = s.size()-1;
    while (start <= end){
        if (!isalnum(s[start])){
            start++;
        }
        else if (!isalnum(s[end])){
            end--;
        }
        else if(tolower(s[start]) != tolower(s[end])){
            return 0;
        }
        else{
            start++;
            end--;
        }
    }
    return 1;
}
```

3) Remove Duplicates from Sorted Array

PAGE No.:
DATE:

Input → [1, 1, 2]
Output → [1, 2], 2

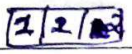
```
int removePuplicates(vector<int>& nums){
    int start = 0, end = 1;

    while(end < nums.size()){
        if(nums[start] != nums[end]){
            start++;
            nums[start] = nums[end];
        }else{
            end++;
        }
    }
    return start+1;
}
```

Dry run:

Input → [1, 1, 2]

start = 0 1
end = 1 2



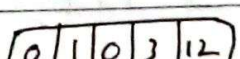
So, unique elements → 2

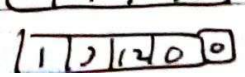
[1|2|2], [1|2] → unique array

1 != 1 ✗
0

1 != 2 ✓
0

4) More Zeros

Input → 

Output → 

```
int start = 0; end = 0;
while (end < nums.length) {
    if (nums[end] != 0) {
        swap(nums[start], nums[end]);
        start++;
    } else {
        end++;
    }
}
```

Q6)

Q5) Squares of Sorted Array

Input → [-4 | -1 | 0 | 3 | 10]

Output → ~~[16 | 1 | 0 | 9 |]~~ [0 | 1 | 9 | 16 | 100]

```
vector<int> SortedSquares(vector<int>& nums) {
    vector<int> ans(nums.size());

    int start = 0;
    int end = nums.size() - 1;
    int ptr = nums.size() - 1;

    while (start <= end) {
        long long ss = nums[start] * nums[start];
        long long es = nums[end] * nums[end];

        if (ss > es) {
            ans[ptr] = ss;
            start++;
        }
        else {
            ans[ptr] = es;
            end--;
```

PAGE No.:
DATE:

```
        ptn--;
    }
    return ans;
```

(86) Merge Sorted array:

Input → nums 1 [1|2|3|0|0|0], m = 3
nums 2 [2|5|6], n = 3

```
void merge(vector<int>& nums1, vector<int>& nums2
           int n){

    int index = m+n-1;
    int i = m-1;
    int j = n-1;

    while(i>=0 && j>=0){
        if(nums1[i] >= nums2[j]){
            nums1[index] = nums1[i];
            index--;
            i--;
        }
        else{
            nums1[index] = nums2[j];
            index--;
            j--;
        }
    }
    // remaining elements in nums2 array
```

P-7-0

```
while (j >= 0) {
    nums1[index] = nums2[j];
    index--;
    j--;
}

//remaining elements in nums1 array

while (i >= 0) {
    nums1[index] = nums1[i];
    index--;
    i--;
}
```

Q7) Two Sum II Input Array is Sorted

Input → [2|7|11|15], target = 9

Output → [2|7]

```
vector<int> twoSum(vector<int>& numbers, int target) {

    vector<int> ans;

    int start, end;
    start = 0;
    end = numbers.size()-1;

    while(start<end){
```

PAGE No.:
DATE:

```
if (numbers[start] + numbers[end] == target) {
        arr.push_back(start+1);
        arr.push_back(end+1);
        return arr;
else if (numbers[start] + numbers[end] < target) {
        start++;
    }
    else {
        end--;
    }
}
return {};
}
```

98) Best Time to Buy and Sell Stock

prices = [7 | 1 | 5 | 3 | 6 | 4]

Output → 5

Explanation: Buy on day 2 and sell on day 5
profit → 6 − 1 → 5

```
int maxProfit (vector<int> & prices) {
    int maxprofit = 0; bestbuy = Prices[0];
    for(int i=1; i<prices.size(); i++) {
     if (Prices[i] > bestbuy) {
         maxprofit = max(maxprofit, prices[i]
                                    - bestbuy);
     }
     bestbuy = min (bestbuy, prices[i]);
    } return maxprofit;
}
```

PAGE No.:
DATE:

Dry run: maxprofit = 0 4
besthuy = ~~X~~ 1
i = ~~1 X X X~~ 5

1 > 7 ×

besthuy = 1

5 > 1 ✓

maxprofit = 4

3 > 1 ✓

maxprofit = 4

6 > 1 ✓

maxprofit = 5

4 > 1 ✓

maxprofit = 5 ✓ Ans.

Sliding Window:

1) Longest Substring Without Repeating Characters

Input s = "abcabcbb"
Output → 3 → (abc), bca and cab are also

```
int lengthOfLongestSubstring(string s) {
    unordered_set<char> st;

    int left = 0;
    int maxLen = 0;

for(int right = 0; right < st.size(); right++) {

    while(st.count(s[right])) {
        st.erase(s[left]);
        left++;
    }
    st.insert(s[right]);

    maxLen = max(maxLen, right - left + 1);
}
return maxLen;
}
```

Dry run :

s = abba

ba

right = 0 1 2 3

left = 0 1 2

maxLen = 0 1 2 2

2) Longest Repeating Character Replacement

Input → s = ABAB, k = 2

Output → 4.

PAGE No.:
DATE:

```
int characterReplacement(string s, int k){
    vector<int> count(26,0);

    int left=0;
    int maxfreq=0;
    int maxlen=0;

    for(int right=0; right<s.size(); right++){

        count[s[right]-'A']++;

        maxfreq = max(maxfreq, count[s[right]-'A']);

        while((right-left+1)-maxfreq >k){
            count[s[left]-'A']--;
            left++;
        }

        maxlen=max(maxlen, right-left+1);
    }
    return maxlen;
}
```

Dry run:

ABAB, k=2

left=0
right=0 1 2 3
maxfreq=0 1 2
maxlen=0 1 2 3 4

0>2 ✗
1>2 ✗
1≥2 ✗
3>2 ✗

4 Ans.

PAGE No.:
DATE:

7) Permutation in String

s1 = "ab", s2 = "eidbaooo"
Output → true

s2 contains one permutation of s1 ("ba").

```
bool checkInclusion(string s1, string s2){
    return sint > count1(26,0)
    return sint> count2(26,0);

for(char ch: s1){
    count1[ch-'a']++;
}

for(int i=0; i<s1.size(); i++){
    count2[s2[i]-'a']++;
}

for(int i=s1.size(); i<s2.size(); i++){
    if(count1==count2)
        return true;

    count2[s2[i-s1.size()]-'a']--;
    count2[s2[i]-'a']++;
}

return count1==count2;
}
```

PAGE No.:
DATE:

Tricky Output Ques

Case 1 Swap without temp

s = hello

l = 0 r = 4

0 < 4 ✓ 1 < 3

s[4] = s[0] h

s[0] = s[4] h

hellh

s[3] = s[1] e

s[1] = s[3] e

helch

without temp it will give error on wrong String

Case 2 Skipping Characters without a Bound check

s = " "

l = 0 1

s = 2

0 < 1 true

Case 3

PAGE No.:
DATE:

Case 3 Pair sum on an unsorted array

arr = [8 | 1 | 5 | 3], target = 8 (0 1 2 3)

l = 0

r = 3 ~~2~~ 1 0

0 < 3 ✓

sum = 8 + 3 ≠ 11

11 == 8 ×

0 < 2 ✓

sum = 13

13 == 8 ×

0 < 1 ✓

sum = 9

9 == 8 ×

Output

Not found and array is not sorted too.

Case 4 l <= r in pair sum

arr = [2 | 5 | 7], target = 10 (0 1 2)

l = 0 1

r = 2 1

0 < 2 ✓ | 1 < 2 ✓ 5+5 = 10

9 < 10 so, | 5 + 7 = 12 > 10

Output | but output should be in pair

PAGE No.:
DATE:

Case 5 More Zeros, Half Done

arr = [0|1|0|3|12]

write = 0 1 2 3

read = 0 1 2 3 4 5

1 != 0 ✓

a[0] = a[1]

a[0] = 1

[1|1|0|3|12]

0 != 0 ✗

3 != 0 ✓

a[1] = a[3]

a[1] = 3

[1|3|0|3|12]

1 != 0

a[2] = a[4]

a[2] = 12

[1|3|12|3|12]

Output: [1|3|12|3|12], write = 3

Case 6 Sort colors (Dutch National Flag) with one extra mid++

PAGE No.:
DATE:

arr = [1|1|0]

low = 0
mid = 0 x 2
high = 2 1

0 <= 2 ✓
a(mid) == 0 x
a(mid) == 1 ✓

1 <= 2 ✓
a(mid) == 0 x
a(mid) == 1 x

Output → [1|0|2], the problem is extra mid++

Case 7 Window Max that starts from 0

arr = [4|-4|-7|-1]
0 1 2 3

k = 2
sum = 0 - 4 - 4 - 4 - 1
max = 0
i = 0 1 2 3 4
for i=0, sum = -4

0 >= 1 x

for i = 1, sum = -6

1 >= 1 ✓

max = 0
sum = -6 - a[0] → -2

for i = 3
sum = -8
3 >= 1 ✓
sum = -8 - a[2]
for i = 2, sum = -9
-8-(-1) → -1
2 >= 1 ✓
max = 0 (-2)
sum = -9 - (-1)
sum = -7

> sum = -6 Ans.

Output
max kivalue
INT_MIN hoga 0 max kivalue

PAGE No.:
DATE:

Case 8 shrinking with if instead of while

s = "abba"

l = 0 x 2

best = 0

r = 0 x 2 3

for r = 0, if x

for r = 1, if x

so, so,

a ab

best = max(0, 1);

best = 1

best = max(1, 2)

best = 2

for r = 2, if ✓ again if ✓

~~xx~~ ~~xxx~~ ba

best = max(2, 2);

best = 2

Output

2

Case 9 Substring () and window length

s = racecar

0 1 2 3 4 5 6

l = 1, r = 5

Output

~~aaccc~~

aacca

5

~~aaccc~~

exception

PAGE No.:
DATE:

case 10 Comparing Two Const Arrays

Output
[redacted] false
[redacted] false
true

.equals() and == reference compare karte hain.

Code Modification Challenge

In VS Code.